AI BASED PERSONAL FINANCE MANAGER

Design and Implementation

Department of Computer Science and Engineering, TKR College of Engineering & Technology

Batch C11 (CSE-C) · Academic year [ACADEMIC YEAR]

Table 1: Project team

S.No.	Student name	Roll number
1	Mekala Kalyan	23K91A05F0
2	Rajanala Sai Hemanth	23K91A05K3
3	Middala Surya Teja Reddy	23K91A05F2
4	Manga Akash	23K91A05E5

Project guide: N Padmavathi

Chapter and section numbers quoted in the text (for example Chapter 8 or Section 6.2) refer to the full project report, 17.Final Document. Figures and tables in this document are numbered from 1.

1. DESIGN

This chapter describes the architecture and workflow, the data flow, the UML and database design, the core algorithms, the modules, the API and the user interface. The diagrams were drawn from the code in `backend/`, `services/`, `ml/`, `database/` and `frontend/src/`. FINJARVIS is a single-role, multi-user web application: every registered user sees only their own records, and all state lives in one SQLite file. The design is stated as built, including where it differs from the batch abstract (Flask and MySQL, loan tracking, scheduled alerts).

1.1 Design goals and architecture

Five goals shaped the design. **Every number on screen is computed from the user's own records**: the dashboard, the forecast, the health score and the assistant call the same service functions over the same tables. **The models are simple enough to inspect**: a TF-IDF and logistic-regression categoriser, a least-squares trend and a six-term weighted sum can each be re-computed by hand, which is why none of the heavier models of the base

paper [1] was built. **One code path serves two front ends**: the Streamlit pages and the React console call the same `services/` and `ml/` modules, and the FastAPI layer holds no business rules. **Per-user isolation**: every table carries a `user_id` and every service query filters on it. **Zero-configuration deployment**: SQLite, a bundled sample file and a seeded demo account run on a laptop with one launcher script.

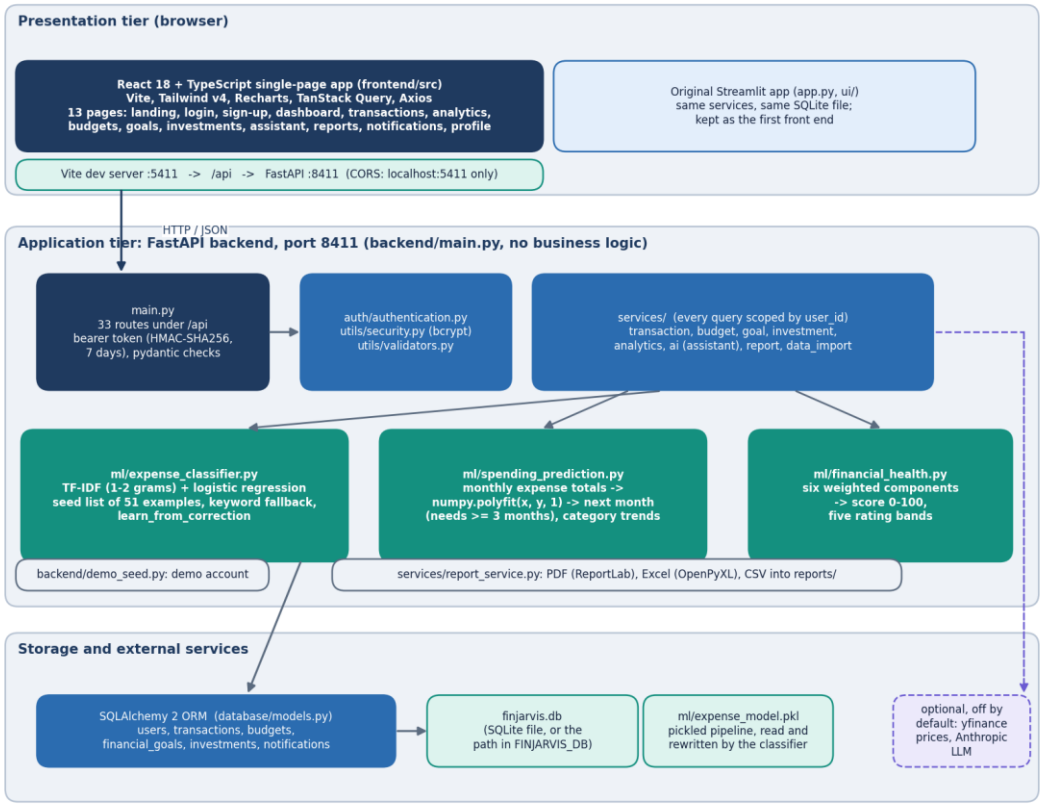


Figure 1: Layered architecture of FINJARVIS: React single-page app, Vite proxy, FastAPI application with 33 routes, service layer, ML modules, SQLAlchemy and SQLite.

The React console on port 5411 reaches the FastAPI application on 127.0.0.1:8411 through the Vite development proxy (`/api` is forwarded), so the browser sees one origin; cross-origin calls are allowed from `localhost:5411` and `127.0.0.1:5411` only. The application exposes 33 routes in `backend/main.py`. Each route validates input with pydantic, resolves the caller through the `current_user` dependency, calls one service or ML function and returns JSON after a `clean` step that converts NumPy and pandas values. The rules live in `services/` and `ml/`, so they can be tested without HTTP. The style is a stateless REST interface [21] over one relational store accessed through an object-relational mapper [22].

The deliberate gaps are worth stating. There is one backend process and one SQLite file, so there is no horizontal scaling. The categoriser is a single pickled file,

`ml/expense_model.pkl`, shared by every account: `learn_from_correction` retrains on the seed list plus one user's history and overwrites it, so one user's correction changes every user's predictions (read from the code, not exercised in a multi-user test). Tokens are home-made HMAC bearer tokens with a 7-day lifetime and no revocation, and there is no scheduler: budget alerts are raised when the dashboard or the notification list is requested.

1.2 High-level workflow

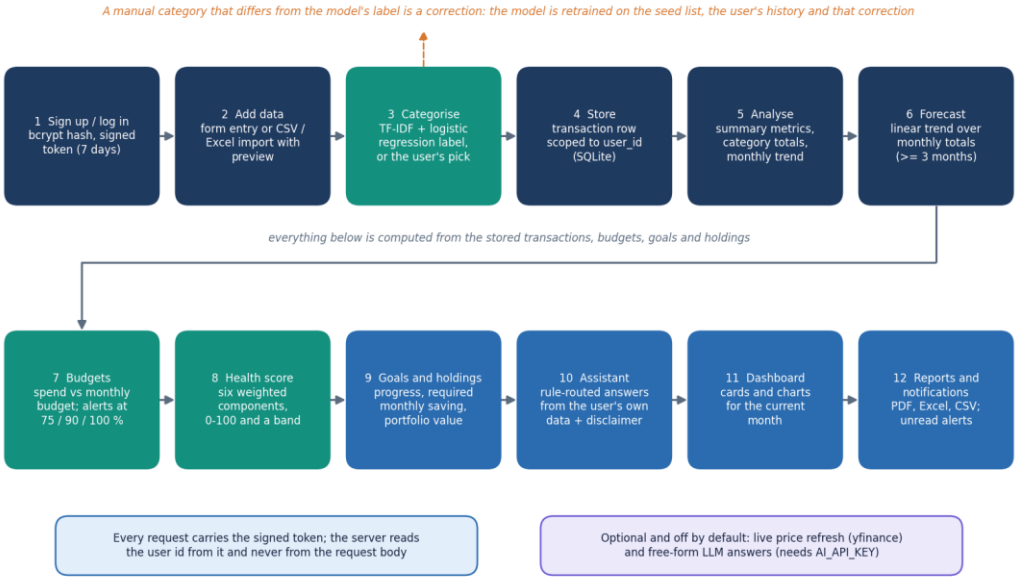


Figure 2: High-level workflow: record or import a transaction, categorise it, update budgets, forecast, score and answer questions.

The workflow has six stages and every stage maps to a route and a page. **Record**: a transaction is typed in or imported from CSV or Excel (preview, then commit). **Categorise**: with 'Auto-detect' the classifier labels the description; a manual category that differs from the model's prediction is stored as chosen and also used to retrain the model, so the classifier is called for every new transaction. **Watch**: budgets are compared with the month's spending and a notification is written when a category crosses 75, 90 or 100 % of its budget. **Forecast**: a straight line over the monthly expense totals, refused below three months of history. **Score**: the dashboard turns the month's metrics, budgets, holdings and goals into the 0-100 health score. **Ask and export**: the assistant answers from the user's records, and the reports routes produce PDF, Excel or CSV files.

1.3 Data flow diagrams

1.3.1 Level 0: context

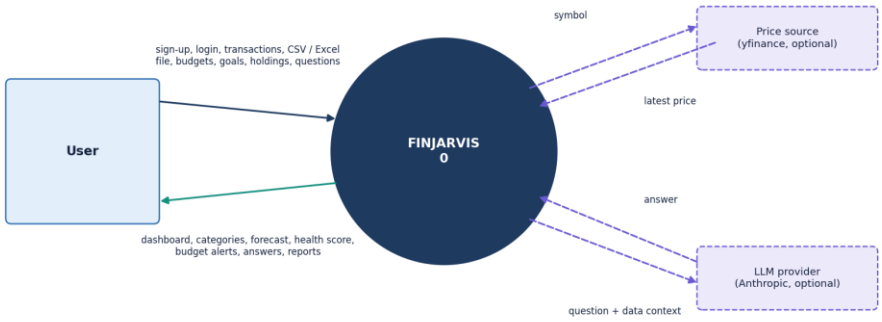


Figure 3: Data flow diagram, level 0: FINJARVIS as one process between the user, the optional LLM provider and the market-data source.

At level 0 FINJARVIS is one process with three external entities. The **user** supplies credentials, transactions, files, budgets, goals, holdings and questions, and receives the dashboard, forecasts, alerts, answers and report files. The **LLM provider** is optional: with `AI_API_KEY` set, a question the rule layer cannot match is sent with a short summary of the user's figures; without a key nothing leaves the machine. The **market-data source** (`yfinance`) is also optional and used only when the user refreshes a holding's price.

1.3.2 Level 1: main processes

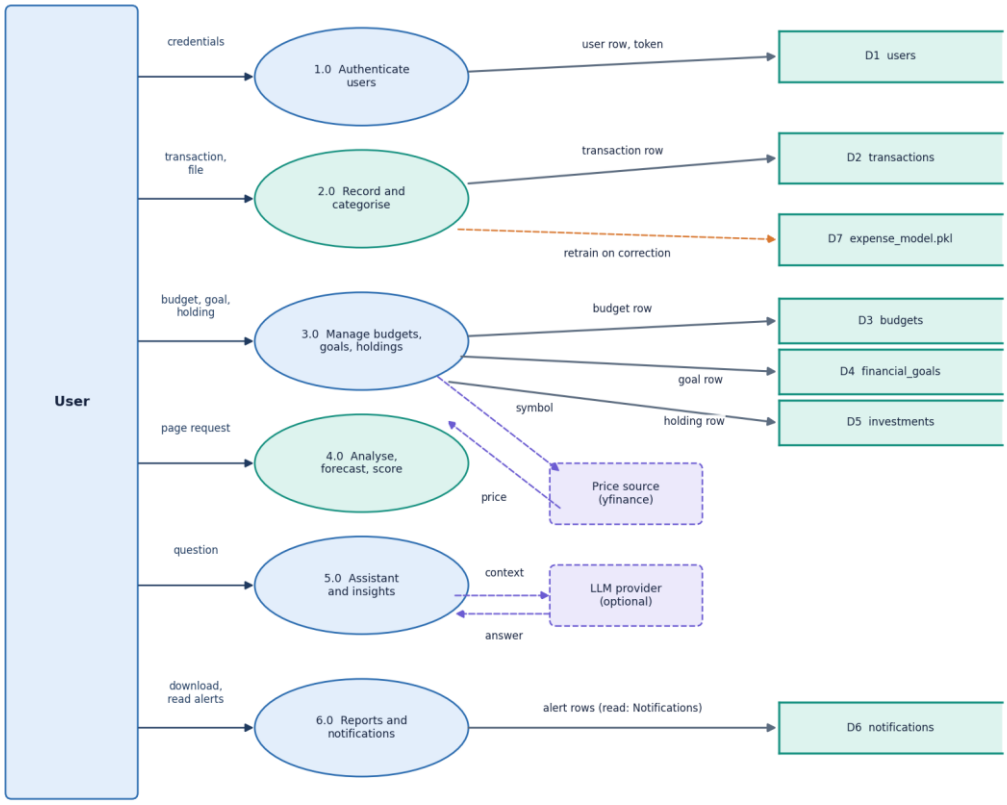


Figure 4: Data flow diagram, level 1: the main processes and the six data stores.

Level 1 shows the processes that read and write the six data stores. Authentication uses the users store; transaction handling writes transactions and calls the categoriser; budget monitoring reads budgets and transactions and writes notifications; analytics reads transactions and holdings and calls the forecast and health modules; the assistant reads the same stores through the analytics functions; reporting writes files only.

1.3.3 Level 2: adding a transaction

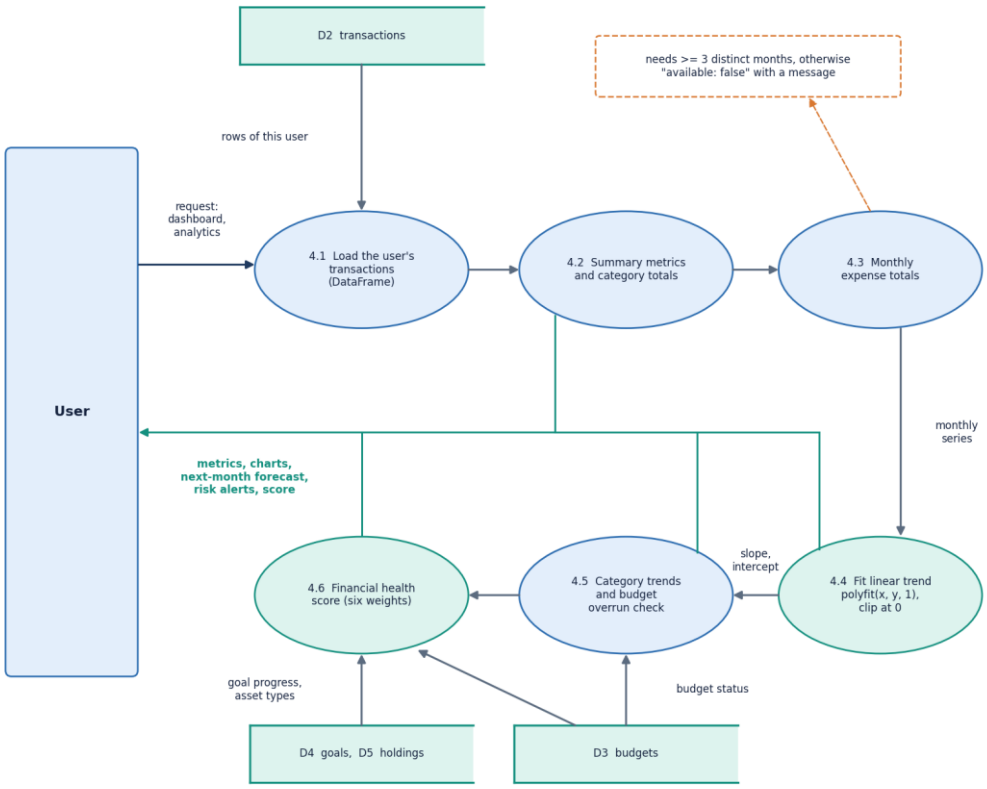


Figure 5: Data flow diagram, level 2: adding a transaction (validation, categorisation, learning, storage).

The transaction process holds the main input rules. The description must not be blank and the amount must be above zero (HTTP 400 otherwise). 'Auto-detect' or an empty category takes the model's label, any other value is kept. If the user's value differs from the model's prediction, the model is retrained on the seed plus the user's history plus the new pair; a retraining failure is swallowed so that it never blocks the insert. The file import follows the same path row by row, except that a bad row is skipped with a warning instead of failing the file.

1.4 UML design

1.4.1 Class diagram

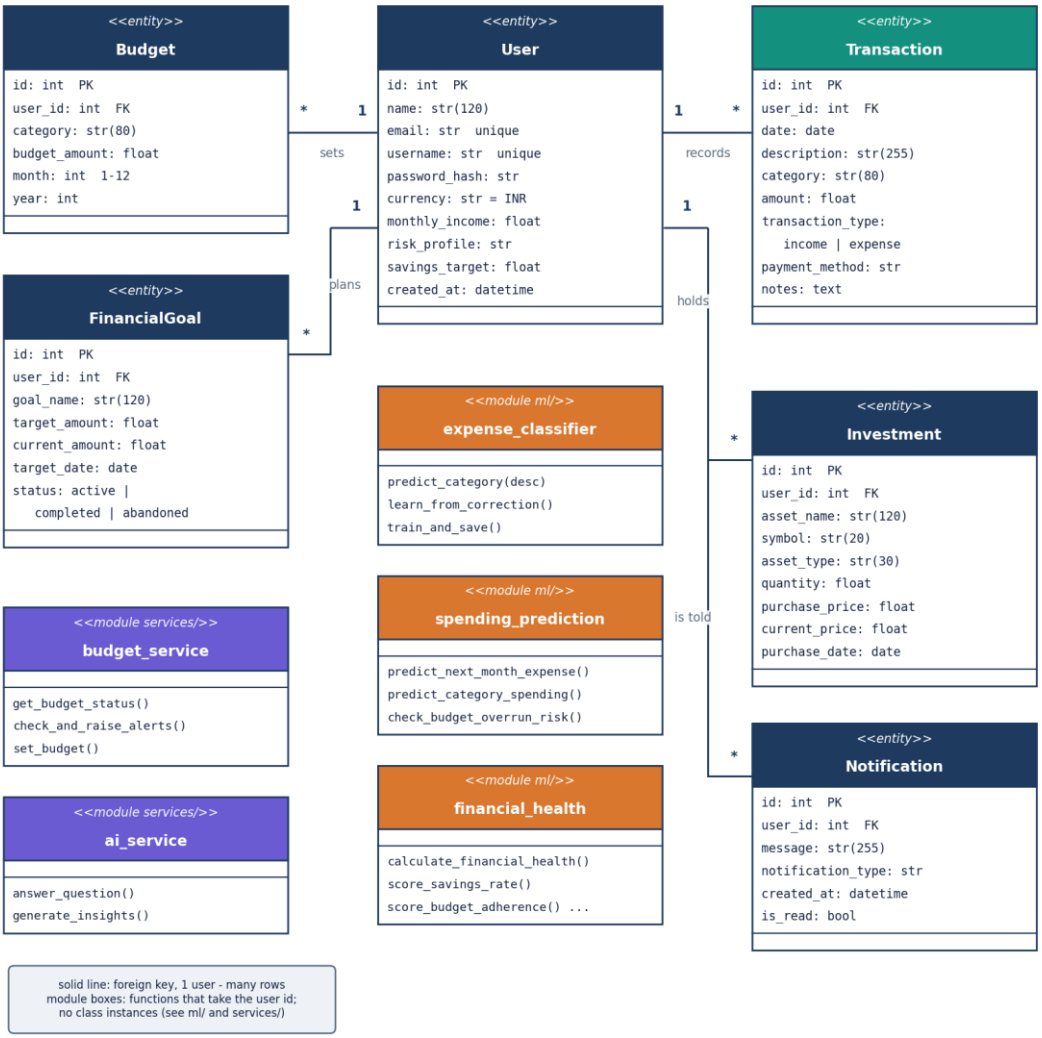


Figure 6: UML class diagram: the six ORM classes, the service functions and the three ML modules.

Six SQLAlchemy classes map to six tables [22]. Behaviour is kept out of the model classes and placed in `services/` as functions that open a session through `get_session`, which commits on success and rolls back on any exception. The only other classes are the pydantic input models of the API (`SignupIn`, `LoginIn`, `TransactionIn`, `BudgetIn`, `GoalIn`, `InvestmentIn`, `AskIn` and others), which carry type and range limits, and the small `AuthResult` dataclass. The ML modules are plain functions; the categoriser holds a scikit-learn `Pipeline` of a `TfidfVectorizer` and a `LogisticRegression`.

1.4.2 Sequence diagram

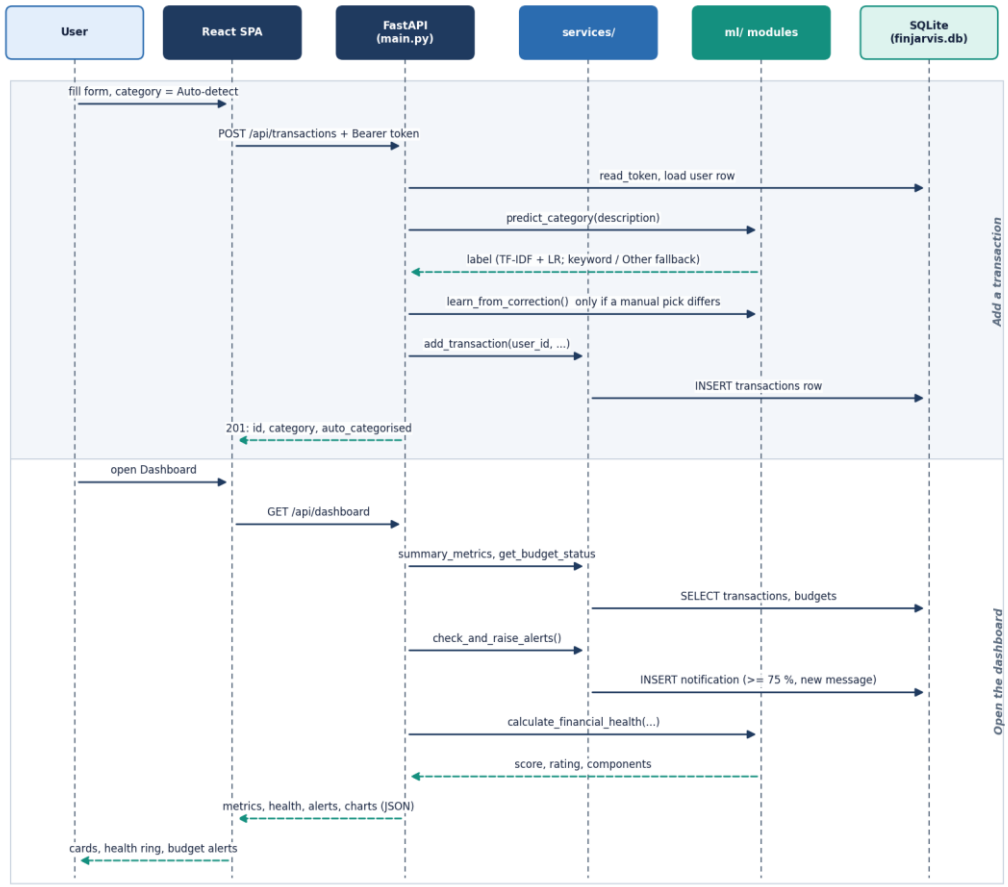


Figure 7: Sequence diagram of the main flow: login, add a transaction with auto-categorisation, then read the dashboard.

The browser posts the login form; the server checks the bcrypt hash and returns a signed token that the console attaches to every later call. A new transaction passes the token check, the field validation, the classifier call and, when the user overrides the label, the retraining attempt. The dashboard request then recomputes the month's metrics, budget status, health score and chart data in one call and raises any budget alert not yet raised.

1.4.3 Use-case diagram

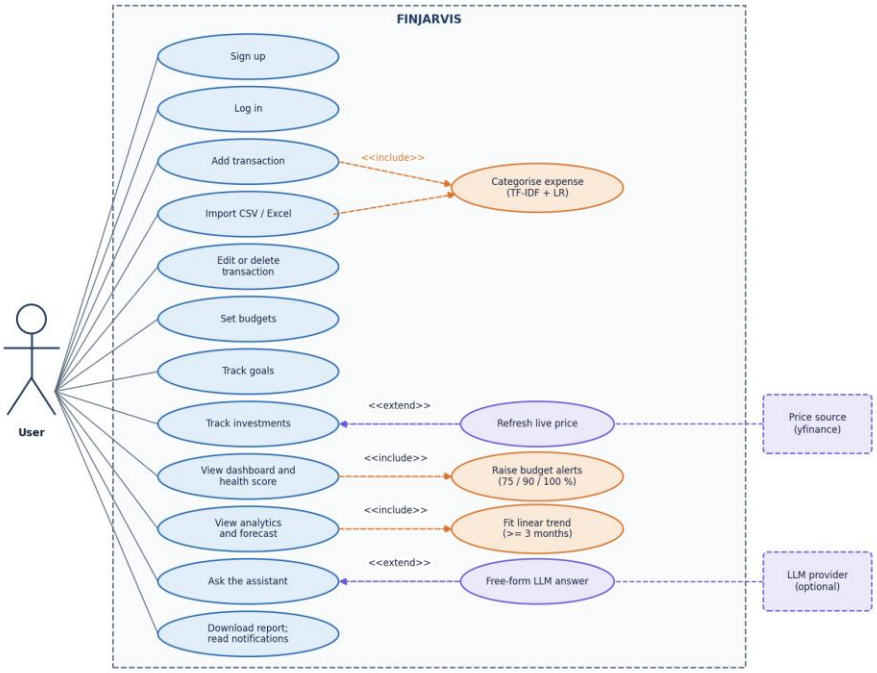


Figure 8: Use-case diagram: the registered user, the demo visitor and the optional external services.

There is one human role, the registered user, and one visitor role: a person without an account can open the landing and login pages and sign in to the fictional demo account. A registered user can manage transactions, import files, set budgets and goals, record holdings, view analytics, ask the assistant, download reports and read notifications. The LLM provider and market-data source are secondary actors. There is no administrator role.

1.5 Database design

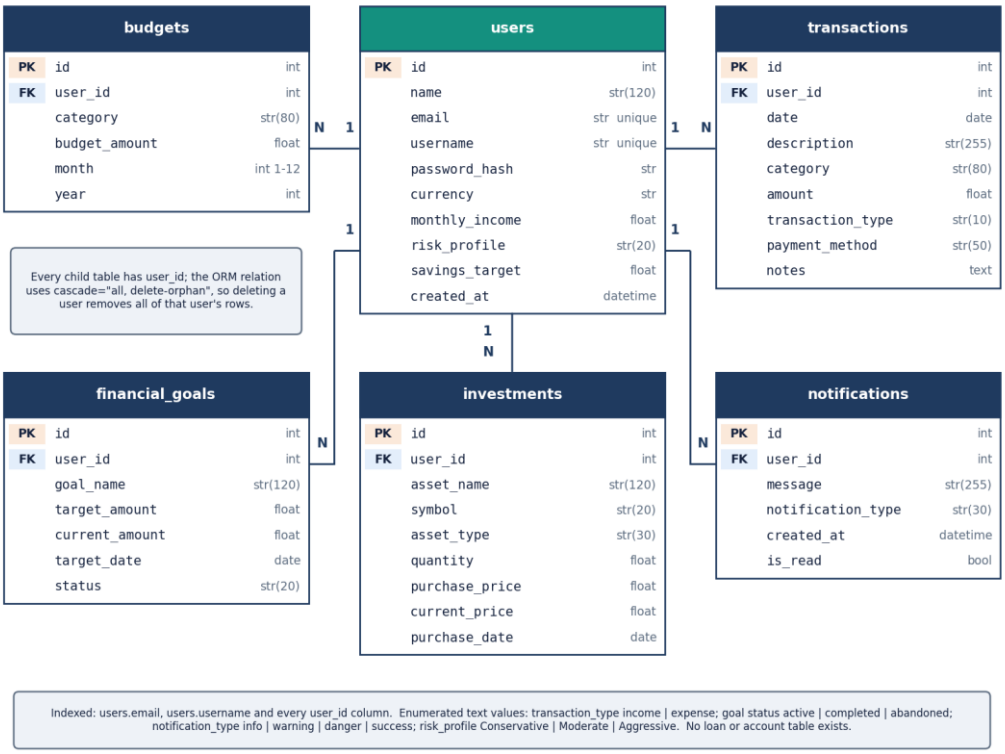


Figure 9: Entity-relationship diagram of the six tables.

The schema has six tables in one SQLite file, `finjarvis.db`, created at start-up by `init_db`; the path can be redirected with `FINJARVIS_DB` (used by the tests and screenshot scripts). A user has many transactions, budgets, goals, investments and notifications; each child row has an indexed, non-null `user_id` foreign key and is removed with its user through the ORM cascade (no route deletes accounts).

Table 2: The six tables and their columns.

Table	Columns	Keys, defaults
users	id, name(120), email(255), username(80), password_hash(255), currency, monthly_income, risk_profile, savings_target, created_at	PK id; unique and indexed email and username; currency INR; risk profile Moderate
transactions	id, user_id, date, description(255), category(80), amount, transaction_type, payment_method, notes	PK id; FK user_id (indexed); type income or expense; payment method Other
budgets	id, user_id, category(80), budget_amount, month, year	PK id; FK user_id (indexed); month 1-12; one row per user, category, month and year is enforced in code (`set_budget` updates the existing row), not by a constraint
financial_goals	id, user_id, goal_name(120), target_amount, current_amount, target_date, status	PK id; FK user_id (indexed); status active, completed or abandoned; current amount 0
investments	id, user_id, asset_name(120), symbol(20), asset_type(30), quantity, purchase_price, current_price, purchase_date	PK id; FK user_id (indexed); asset types Stock, Mutual Fund, ETF, Gold, Fixed Deposit, Other

Table	Columns	Keys, defaults
notifications	id, user_id, message(255), notification_type, created_at, is_read	PK id; FK user_id (indexed); type info, warning, danger or success; unread by default

Three design points deserve a note. Money is stored as floating-point numbers, adequate for a display tool but not for ledger accounting. Monthly queries use string bounds such as `YYYY-MM-31`, which SQLite compares correctly but a stricter database would reject for shorter months. Alerts are de-duplicated by identical message text, so a budget that crosses the same threshold again in a later month raises no new alert.

1.6 Core algorithm design

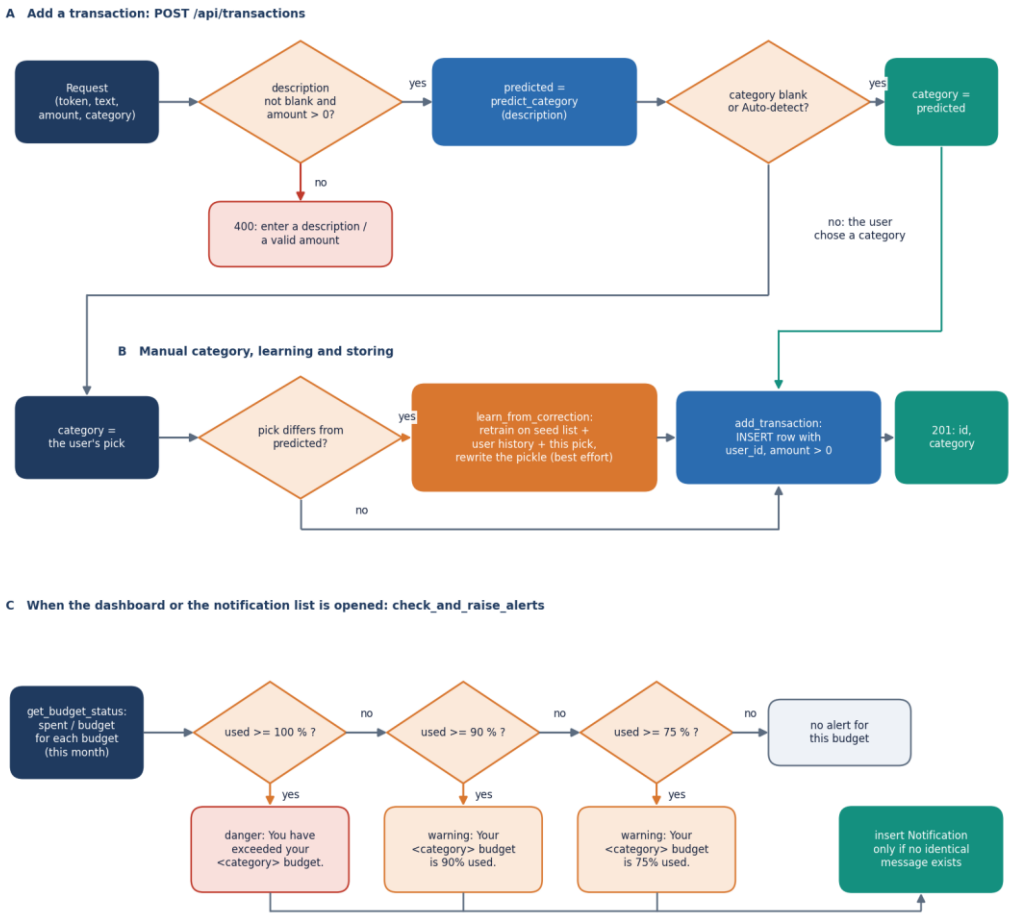


Figure 10: Flowchart of the core algorithms: categoriser, forecast, health score and budget alerts.

Three small algorithms carry the intelligence of the product: the **categoriser** (a description to one of eleven labels), the **forecast** (monthly expense totals to a next-month figure) and the **health score** (the month's records to a 0-100 number and the weakest

area). The flowchart shows how routes call them; Chapter 6 gives formulas and parameters.

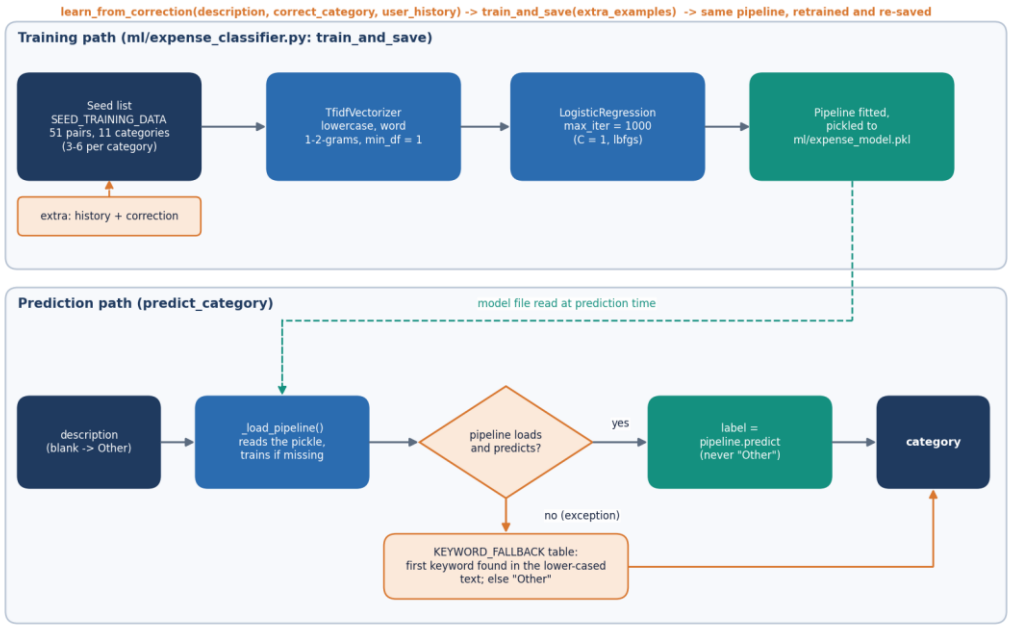


Figure 11: Categoriser pipeline: description, TF-IDF, logistic regression, label, optional correction and retraining.

The categoriser lowercases the text and builds word unigrams and bigrams; TF-IDF weighting [15] gives each term a weight that falls with how many descriptions contain it; a logistic-regression classifier [14] maps the weights to a probability per label and the highest wins. The prediction order is fixed: model, then a keyword table if the model fails, then 'Other'; a model trained on 51 examples never outputs 'Other'. The design choice that matters most is text with no known words: the feature vector is all zeros and the label comes from the intercepts alone. Chapter 8 measures what that costs.

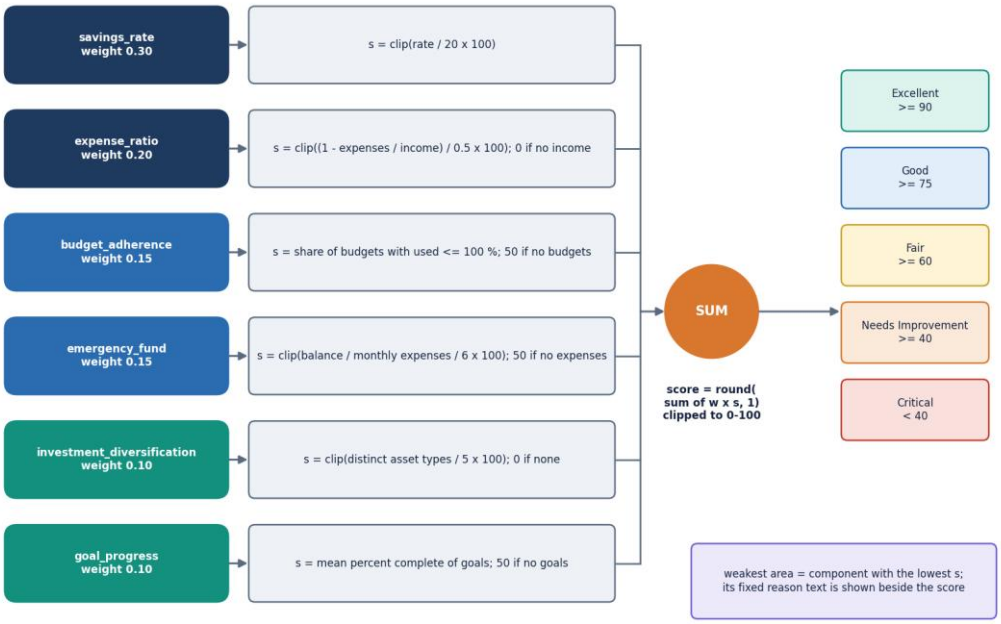


Figure 12: Financial health score model: six weighted components, clipped sum, rating band.

The health score is a weighted sum of six sub-scores, each clipped to 0-100: savings rate (weight 0.30), expense ratio (0.20), budget adherence (0.15), emergency fund (0.15), investment diversification (0.10) and goal progress (0.10). The weights are chosen by the team, not fitted to data. The result carries a rating band (Excellent from 90, Good from 75, Fair from 60, Needs Improvement from 40, otherwise Critical) and the name and reason of the lowest component, so the number is always explained. The forecast follows the same philosophy: a straight line with an explicit refusal below three months is more honest than a deeper model that would fit noise; Chapter 8 tests that claim and shows the refusal threshold is too low for the method.

1.7 Module design

Each file has one reason to change. Line counts were measured from the repository when this report was built.

Table 3: Modules, sizes and responsibilities.

Module	Lines	Responsibility
`backend/main.py`	666	FastAPI application: 33 routes, token helpers, pydantic input models, `current_user` dependency, JSON clean-up
`backend/demo_seed.py`	138	Demo account and demo-data loader (shifted sample CSV plus synthetic earlier months)
`auth/authentication.py`	82	`signup` and `login` with validation and duplicate checks
`utils/security.py`, `utils/validators.py`	56	bcrypt hashing through passlib; e-mail, username, password-strength and amount validators

Module	Lines	Responsibility
`database/models.py`, `database/database.py`	110	Six SQLAlchemy models; SQLite engine, session factory and `get_session` context manager
`services/transaction_service.py`	94	Add, edit, delete and filter transactions, every query scoped by `user_id`
`services/budget_service.py`	96	Budget upsert, usage per category, 75/90/100 % alert notifications
`services/goal_service.py`, `services/investment_service.py`	68	Goals with required monthly saving; holdings, portfolio profit and loss, optional price refresh
`services/analytics_service.py`	100	Monthly metrics, category totals, monthly trend, range overview
`services/ai_service.py`	227	Rule-routed assistant (five handlers), insights, optional LLM layer, disclaimer
`services/data_import_service.py`, `services/report_service.py`	84	CSV/Excel import with row-level warnings; PDF, Excel and CSV reports
`ml/expense_classifier.py`	115	TF-IDF and logistic-regression categoriser, 51-pair seed, keyword fallback, `learn_from_correction`
`ml/spending_prediction.py`	91	Linear-trend forecast of next month's expenses, per-category forecast, budget-overrun check
`ml/financial_health.py`	107	Six sub-scores, weights, rating bands and weakest-area explanation
`scripts/smoke_test.py`	170	49-check end-to-end test against a throw-away database
`scripts/run_experiments.py`, `scripts/measure_api.py`	354	Offline experiments (categoriser, forecast, health score, assistant, import) and API latency
`frontend/src/`	6,959	React 18 and TypeScript console: 13 pages plus a not-found page, UI kit, charts, API client, auth and theme providers
`app.py`, `ui/`	938	The original Streamlit front end over the same services (kept, not extended)
`tests/`	359	38 pytest tests (auth 9, budget 6, financial health 6, ML 9, transactions 8)

1.8 API design

All routes live under `/api`; reads are GET, actions POST, edits PUT and removals DELETE. Every route except `health`, `public/showcase`, `auth/signup` and `auth/login` needs an `Authorization: Bearer` token. Failures return the FastAPI `detail` string, which the console shows verbatim, so messages are written for the user. 401 means the token is missing, tampered or expired (the console returns to login), 404 means the object does not exist or belongs to another user, 409 a duplicate e-mail or username, 400 a well-formed but unacceptable request, and 422 failed type validation.

Table 4: The 33 routes of the FINJARVIS API.

Method	Path	Auth	Purpose	Errors
GET	/api/health	-	Service status and whether the LLM key is set	-
GET	/api/public/showcase	-	Landing-page figures from the fictional demo account, plus one assistant answer	404 no demo account
POST	/api/auth/signup	-	Validate, hash the password with bcrypt, create the account	400 validation; 409 duplicate
POST	/api/auth/login	-	Username or e-mail plus password; returns a signed token and the user	401
GET	/api/auth/me	token	The signed-in user's profile	401
GET	/api/meta	token	Categories, payment methods, asset types, currencies, risk profiles, goal presets	401

Method	Path	Auth	Purpose	Errors
PUT, POST	/api/profile, /api/profile/demo-data	token	Update the profile; load the shifted sample data, budgets, goals and holdings	400; 409; 422
GET	/api/dashboard	token	Month metrics, health score, budget alerts and six chart datasets	401
GET, POST	/api/transactions	token	Filter, sort and total; create ('Auto-detect' uses the classifier, a differing manual category triggers learning)	422 bad date; 400
PUT, DELETE	/api/transactions/{id}	token	Edit selected fields; delete one transaction	400; 404
POST	/api/transactions/predict-category	token	Category the model would assign to a description	401
POST	/api/transactions/import/preview, /commit	token	Parse and clean a CSV/Excel file and label blanks; insert the cleaned rows	400 empty, missing columns, no valid rows
GET	/api/analytics	token	Overview, category split, trend, next-month and category forecasts, overrun risks	400 start after end
GET, POST	/api/budgets	token	Usage per category for a month; create or update (upsert by category, month, year)	422; 400 amount
DELETE	/api/budgets/{id}	token	Delete a budget	404
GET, POST	/api/goals	token	Goals with percent complete and required monthly saving; create a goal	400
PUT, DELETE	/api/goals/{id}	token	Set the saved amount (completes at target); delete a goal	404
GET, POST	/api/investments	token	Holdings and portfolio totals; add a holding	400
POST, DELETE	/api/investments/{id}/refresh, /{id}	token	Try a live price through yfinance (degrades to a message); delete a holding	404
GET, POST	/api/assistant/insights, /ask	token	Rule-based insights plus the disclaimer; answer a question from the user's own data	401
GET	/api/reports/{pdf|excel|csv}	token	Download a monthly report (CSV: all transactions)	400 nothing to export; 500
GET, POST	/api/notifications, /read-all	token	Raise budget alerts, then list notifications and the unread count; mark all read	401

`GET /api/public/showcase` needs no login so that the landing page can show live figures; it reads only the fictional demo account. Import `preview` and `commit` parse the same upload twice, so the user sees the cleaned rows and warnings before deciding.

1.9 User interface design

The console has 13 pages and a not-found page: a public landing page, log-in and sign-up pages, and ten pages behind the login. A route guard sends signed-out visitors to the login page, and a 401 from the API clears the stored token. The interface is dark with a light theme, one gradient accent and a collapsible sidebar; amounts use Indian digit grouping and a compact K, L and Cr notation on charts.

Table 5: Pages, routes and the endpoints they use.

Page	Route	Endpoints used
Landing	/	GET /api/public/showcase
Log in, Sign up	/login, /signup	POST /api/auth/login, POST /api/auth/signup
Dashboard	/app	GET /api/dashboard
Transactions	/app/transactions	GET, POST /api/transactions, PUT, DELETE /api/transactions/{id}, POST predict-category, import preview and commit
Analytics	/app/analytics	GET /api/analytics

Page	Route	Endpoints used
Budgets	/app/budgets	GET, POST /api/budgets, DELETE /api/budgets/{id}
Goals	/app/goals	GET, POST /api/goals, PUT, DELETE /api/goals/{id}
Investments	/app/investments	GET, POST /api/investments, POST refresh, DELETE
AI Assistant	/app/assistant	GET /api/assistant/insights, POST /api/assistant/ask
Reports	/app/reports	GET /api/reports/{pdf|excel|csv}
Notifications	/app/notifications	GET /api/notifications, POST read-all
Profile	/app/profile	GET /api/auth/me, PUT /api/profile, POST /api/profile/demo-data

Every page opens with its purpose in one sentence, long actions show a busy state and finish with a toast, money is formatted the same way everywhere, and API errors appear as a sentence next to the control that caused them. Chapter 8 shows the working pages with real data; the screens below illustrate the design itself.

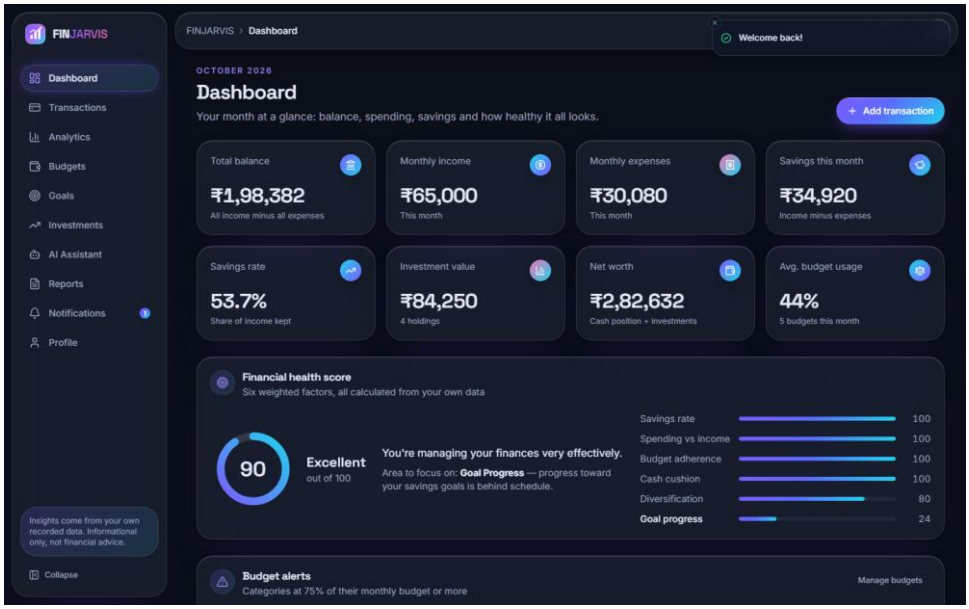


Figure 13: Dashboard: month metrics, financial health score with its six components, and budget alerts, all computed from the demo account's own records.

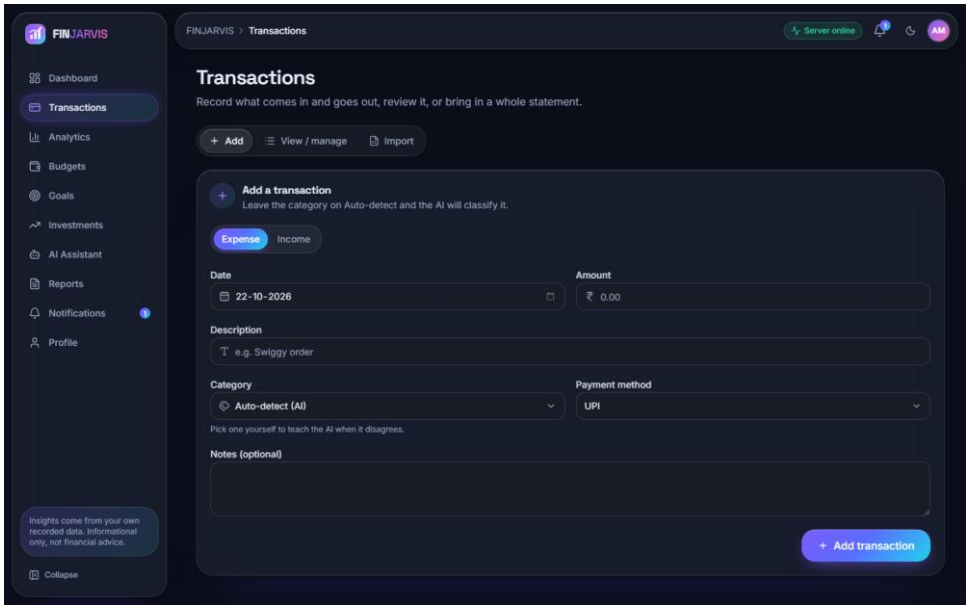


Figure 14: Add-transaction form with 'Auto-detect' category: the model's label is shown before the row is saved.

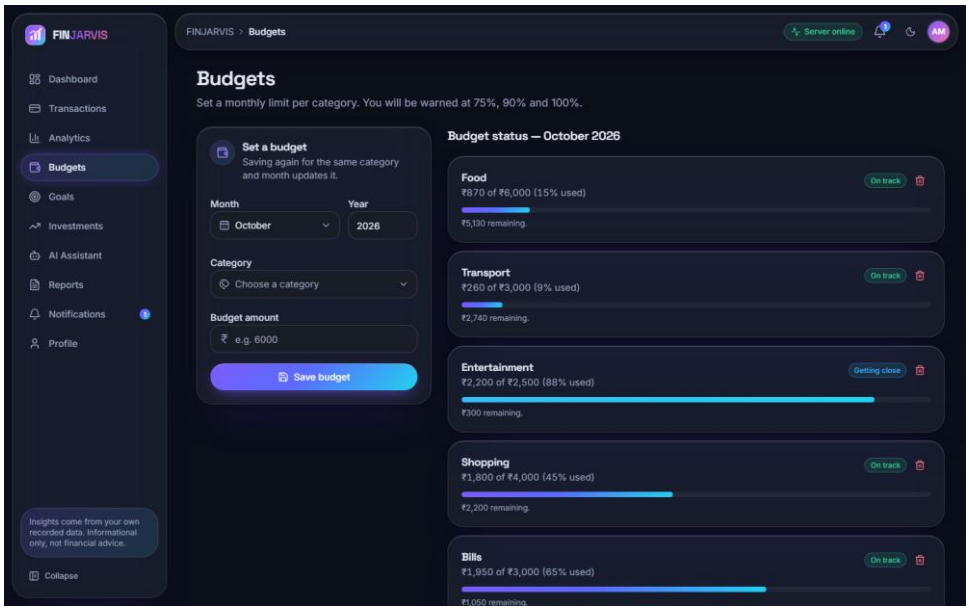


Figure 15: Budgets page: usage per category against the monthly budget with the 75, 90 and 100 % alert bands.

1.10 Design decisions and trade-offs

The table records the main decisions, what each bought and what it cost, so that the limitations in Chapter 9 trace back to a choice.

Table 6: Principal design decisions.

Decision	Reason	Cost
FastAPI, React and SQLite instead of the abstract's Flask and MySQL	The existing service code was Python and database-agnostic; SQLite needs no server; FastAPI gives typed input models and automatic documentation	A deviation from the registered abstract; single-file, single-writer storage; no migrations

Decision	Reason	Cost
Logistic regression on TF-IDF instead of BERT	Trains in milliseconds on a CPU laptop, needs no GPU and no labelled corpus of thousands; predictions are inspectable	A 51-example seed gives 18.2 % accuracy on unseen merchants; no pre-trained language knowledge
Straight-line trend with a three-month minimum	Honest about small histories and trivial to explain	Not better than a plain average below about seven months in the simulation; no seasonality, no uncertainty range
Weighted explainable health score	Every point of the score can be traced to a component and a reason	Weights are chosen, not fitted; savings rate and expense ratio are strongly correlated and carry half the score
Rule-routed assistant with an optional LLM	Answers are computed from the user's own data and work offline	Only five question patterns; 14 of 20 answerable test questions were handled
Single shared model file	Simple persistence, no per-user model management	One user's correction changes every user's predictions
Home-made HMAC token instead of a standard JWT library	No extra dependency; short and auditable	No refresh or revocation; the default secret must be replaced through `APP_SECRET_KEY`
Alerts raised on read, not on a timer	No scheduler process to run or fail	No alert if the user never opens the dashboard or notifications; the abstract's scheduled digests are not provided

Outside the delivered design: no anomaly detector (objective 4), no time-series model beyond the linear trend (objective 3), no reinforcement-learning budget recommender, no loan or debt tracker, no bank or SMS integration and no scheduled digest. Budget advice is rule-based: the assistant suggests trimming the top one or two categories by 10 to 15 % and the budget screen alerts at fixed thresholds.

2. IMPLEMENTATION

2.1 Application architecture

The product runs as two processes. The **backend** is `backend/main.py`, a FastAPI application served by uvicorn on 127.0.0.1:8411. At start-up its lifespan handler calls `init_db()`, which creates the six tables if they do not exist, and `demo_seed.ensure_demo_user()`, which creates the demo account once; a failure to seed is printed and does not stop the server. The **frontend** is a React 18 and TypeScript single-page application built with Vite and served on port 5411; the Vite server proxies `/api` to the backend, so the browser makes only same-origin calls.

The console uses React Router 7 for routes and the sign-in guard, Axios for the API client, TanStack Query for fetching and caching, Recharts for charts, Tailwind CSS 4 with Radix UI for styling and accessible controls, and framer-motion, GSAP and Lenis for motion. The token is kept in local storage under `finjarvis-token`; an Axios interceptor adds it to every request and clears it on a 401. A helper turns any error into a sentence: the API's `detail` text, a field-level message list, or a hint to start the backend when nothing responds. On the server the `clean` helper makes service output JSON-safe (NumPy scalars to numbers, NaN and infinity to null, data frames to records, dates to ISO strings); it is the only translation layer, so the Streamlit pages use the services unchanged.

```
def current_user(authorization: str | None = Header(default=None)) -> SimpleNamespace:
    token = (authorization or "").removeprefix("Bearer ").strip()
    uid = read_token(token) if token else None
    if uid is None:
        raise HTTPException(401, "Your session has expired. Please log in again.")
    with get_session() as session:
        u = session.query(User).filter(User.id == uid).first()
        if not u:
            raise HTTPException(401, "This account no longer exists. Please log in again.")
        return SimpleNamespace(**user_dict(u))
```

Every protected route takes `user=Depends(current_user)` and passes `user.id` to the service layer, which filters every query on it: this dependency is the whole authorisation model (no roles, only ownership).

2.2 API design and key code

The 33 routes are listed in Chapter 5. The create-transaction route is where the categoriser, the learning step and the validation meet:

```
@app.post("/api/transactions", status_code=201)
def create_transaction(body: TransactionIn, user=Depends(current_user)):
```

```
    if not body.description.strip():
        raise HTTPException(400, "Please enter a description.")
    if not is_valid_amount(body.amount):
        raise HTTPException(400, "Please enter a valid amount greater than zero.")
    chosen = (body.category or "").strip()
    auto = chosen in ("", "Auto-detect")
    predicted = predict_category(body.description)
    if auto:
        category = predicted
    else:
        category = chosen
        if predicted != category:   # a manual pick that differs from the model is a correction
            history = [(t["description"], t["category"]) for t in get_transactions(user.id)]
            try:
                learn_from_correction(body.description, category, history)
            except Exception:
                pass                # learning is best-effort, never blocks adding the
transaction
    txn_id = add_transaction(user.id, body.date, body.description, category, body.amount,
                             body.transaction_type, body.payment_method, body.notes)
    return {"id": txn_id, "category": category, "auto_categorised": auto}
```

The response tells the console which category was used and whether it was automatic. Retraining reads the user's entire history on every correction, which is cheap for a few hundred rows and would not scale to a large account.

The forecast function returns a structured refusal instead of a number when the history is too short, and the analytics route passes it on unchanged:

```
def predict_next_month_expense(df: pd.DataFrame) -> dict:
    totals = _monthly_totals(df, "expense")
    if len(totals) < MIN_MONTHS_REQUIRED:
        return {"available": False,
                "message": f"Need at least {MIN_MONTHS_REQUIRED} months of transaction "
                           f"history for a reliable prediction (currently have
{len(totals)})."}
    y = totals.values.astype(float)
    x = np.arange(len(y))
    slope, intercept = np.polyfit(x, y, 1)
    predicted = max(0.0, slope * len(y) + intercept)
    return {"available": True,
            "predicted_next_month_expense": round(predicted, 2),
            "recent_average": round(float(np.mean(y[-3:])), 2),
            "trend": "increasing" if slope > 1 else ("decreasing" if slope < -1 else
"stable"),
            "months_used": len(y)}
```

The health score combines the sub-scores exactly as in Section 6.5.3: the dashboard route builds the six components, takes the weighted sum rounded to one decimal and names `weakest = min(components, key=components.get)`. Budget alerts are raised by `check_and_raise_alerts`, which the dashboard and notification routes call: for each budget of the month it computes the percent used, writes a danger notification at 100 % or more and a warning at 90 % and at 75 %, and skips a message already present. Because the check runs on read, an alert appears the next time the user opens either page.

2.3 Security and error handling

Passwords. A password needs at least eight characters with an upper-case letter, a lower-case letter, a digit and a special character, and is hashed with bcrypt through passlib [20]. Login accepts a username or an e-mail and answers a wrong user and a wrong password with the same message; the sign-up form does reveal a taken e-mail or username (HTTP 409), as registration forms normally do. passlib is incompatible with bcrypt 5, so the requirements pin `bcrypt==4.0.1`.

Tokens. A token is a base64 JSON payload (user id, seven-day expiry) and an HMAC-SHA256 signature made with `APP_SECRET_KEY`, checked with a constant-time comparison; a changed or expired token fails with 401. If the secret is not set the code uses the public development value `finjarvis-dev-secret`, acceptable for a demonstration and not for a deployment. There is no refresh token and no revocation other than changing the secret, and the token sits in local storage, where an injected script could read it.

Data isolation. Every service function filters on the user id, so another user's transaction, budget, goal or holding answers 404 (the smoke test checks edit and delete). The ORM binds query parameters, so text is never concatenated into SQL, and pydantic models bound the input types.

Error handling. `get_session` commits on success and rolls back on any exception; import row errors become warnings; the assistant degrades from language model to rules to fixed help text; a failed price refresh returns a message; a failed report returns HTTP 500 with the reason; retraining and demo-seeding errors are swallowed on purpose so that a secondary feature cannot take down the primary one.

Known weaknesses (repeated in Chapters 7 and 9): no rate limit or lockout on login, a default secret, tokens that cannot be revoked, a model file shared by all users, CORS and proxy set for localhost only, no HTTPS, and the optional language-model layer sends a summary of the user's figures to a third party when a key is set, which a deployment would have to disclose.

2.4 Installation and execution

Table 7: Software requirements.

Item	Version or note
Python	3.11 or newer (3.11.9 used for the measurements)

Item	Version or note
Node.js	20 or newer
Backend packages	fastapi, uvicorn, python-multipart, sqlalchemy 2, pandas, numpy, scikit-learn, bcrypt 4.0.1, passlib, python-dotenv, pydantic 2, yfinance, reportlab, openpyxl, httpx, requests
Optional	`anthropic` and an `AI_API_KEY` for the language-model layer; internet access for price refresh
Frontend packages	react 18, react-router 7, @tanstack/react-query, axios, recharts, tailwindcss 4, radix-ui, framer-motion, gsap, lenis, sonner, vite
Test tools	pytest (38 tests), `scripts/smoke_test.py` (49 checks)

1. Open the product folder and double-click `run.bat`. It checks Python and Node, builds `.venv` and installs the backend packages (rebuilding `.venv` if it was copied from another computer), runs `npm install` for the frontend, starts the backend, waits for `/api/health` to answer and starts the frontend.
2. Open http://localhost:5411 and sign in with the demo account: username `demo` and password `Demo@1234` (fictional data), or sign up with a new account; a new account starts empty and the Profile page has a button that loads the demo data into it.
3. To run the pieces by hand: `python -m venv .venv`, `pip install -r backend/requirements.txt`, `python -m uvicorn backend.main:app --host 127.0.0.1 --port 8411` from the product folder, and `npm install` then `npm run dev` inside `frontend/`.
4. Copy `.env.example` to `.env` to change the ports, to set `APP_SECRET_KEY` to a long random string, or to add `AI_API_KEY`. Set `FINJARVIS_DB` to use another SQLite file.
5. To check an installation run `python -m pytest tests -q` (38 tests) and `python scripts/smoke_test.py` (49 checks, on a throw-away database and a spare port).

Appendix B repeats these instructions with screens. The first start of `run.bat` takes several minutes because of package downloads, and the Streamlit front end still runs with `streamlit run app.py` on port 8501 against the same database file.

3. KEY CODE WALK-THROUGHS

3.1 Categorisation and learning from corrections (`ml/expense_classifier.py`)

The categoriser is a scikit-learn pipeline of a TF-IDF vectoriser over word one- and two-grams and a logistic regression. It is trained from the 51 seed pairs, pickled to `ml/expense_model.pkl` and re-created if the file is missing. Prediction takes 0.84 ms and retraining on the seed takes 5.2 ms. If the model cannot be loaded the function falls back to a keyword table and finally to Other; the model itself never returns Other.

```
def _train_pipeline(descriptions: list[str], labels: list[str]) -> Pipeline:
    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(lowercase=True, ngram_range=(1, 2), min_df=1)),
        ("clf", LogisticRegression(max_iter=1000)),
    ])
    pipeline.fit(descriptions, labels)
    return pipeline


def predict_category(description: str) -> str:
    description = (description or "").strip()
    if not description:
        return "Other"


    pipeline = _load_pipeline()
    if pipeline is not None:
        try:
            return pipeline.predict([description])[0]
        except Exception:
            pass


    lowered = description.lower()
    for keyword, category in KEYWORD_FALLBACK.items():
        if keyword in lowered:
            return category
    return "Other"
```

The route that adds a transaction treats a manual category that differs from the prediction as a correction. It retrains on the seed, the user's whole history and the new pair; failures are swallowed so that adding the transaction never fails.

```
chosen = (body.category or "").strip()
auto = chosen in ("", "Auto-detect")
predicted = predict_category(body.description)
if auto:
    category = predicted
else:
    category = chosen
    if predicted != category:  # a manual pick that differs from the model is a correction to learn
from
        history = [(t["description"], t["category"]) for t in get_transactions(user.id)]
        try:
            learn_from_correction(body.description, category, history)
        except Exception:
            pass  # learning is best-effort, never blocks adding the transaction
```

```
def learn_from_correction(description: str, correct_category: str, user_history: list[tuple[str,
str]]) -> None:
    examples = list(user_history) + [(description, correct_category)]
    train_and_save(extra_examples=examples)
```

3.2 Next-month forecast (`ml/spending_prediction.py`)

Expenses are summed per calendar month and a degree-one least-squares polynomial is fitted over the month index. The next value is the line evaluated one step ahead and clipped at zero. The label is increasing if the slope exceeds 1, decreasing below -1 and stable otherwise, and the recent average is the mean of the last three months. With fewer than three months the function declines. For the demo account it gives INR 39,471 for November against a recent average of INR 36,592.

```
def predict_next_month_expense(df: pd.DataFrame) -> dict:
    totals = _monthly_totals(df, "expense")
    if len(totals) < MIN_MONTHS_REQUIRED:
        return {
            "available": False,
            "message": f"Need at least {MIN_MONTHS_REQUIRED} months of transaction "
                       f"history for a reliable prediction (currently have {len(totals)}).",
        }

    y = totals.values.astype(float)
    x = np.arange(len(y))
    # Simple least-squares linear fit; clip prediction at 0 (can't spend negative money)
    slope, intercept = np.polyfit(x, y, 1)
    next_x = len(y)
    predicted = max(0.0, slope * next_x + intercept)

    return {
        "available": True,
        "predicted_next_month_expense": round(predicted, 2),
        "recent_average": round(float(np.mean(y[-3:])), 2),
        "trend": "increasing" if slope > 1 else ("decreasing" if slope < -1 else "stable"),
        "months_used": len(y),
    }
```

3.3 Financial health score (`ml/financial_health.py`)

Each component is mapped to 0-100 and clipped. The savings rate gets full marks at 20 percent; the expense ratio at half of income or less; emergency cover at six months of expenses; diversification at five asset types. Missing budgets, goals or expenses give a neutral 50. The weighted sum selects one of five bands and the lowest component becomes the explanation. The demo account scores 90.4.

```
def score_savings_rate(savings_rate: float) -> float:
    # 20%+ savings rate = full marks; scales linearly below that; negative = 0
    return _clip((savings_rate / 20.0) * 100)


def score_expense_ratio(monthly_income: float, monthly_expenses: float) -> float:
    if monthly_income <= 0:
        return 0.0
    ratio = monthly_expenses / monthly_income
    # ratio <= 0.5 is excellent, ratio >= 1.0 (spending it all or more) is 0
    return _clip((1.0 - ratio) / 0.5 * 100)


def score_emergency_fund(total_balance: float, monthly_expenses: float) -> float:
    if monthly_expenses <= 0:
        return 50.0
```

```
    months_covered = total_balance / monthly_expenses
    # 6 months of expenses covered = full marks
    return _clip((months_covered / 6.0) * 100)
```

```
total = sum(components[k] * WEIGHTS[k] for k in WEIGHTS)
total = round(_clip(total), 1)

if total >= 90:
    rating, explanation = "Excellent", "You're managing your finances very effectively."
elif total >= 75:
    rating, explanation = "Good", "Your finances are in solid shape with room for fine-tuning."
elif total >= 60:
    rating, explanation = "Fair", "You're on a reasonable track but some areas need attention."
elif total >= 40:
    rating, explanation = "Needs Improvement", "Several areas of your finances need active
attention."
else:
    rating, explanation = "Critical", "Your finances need immediate attention across multiple
areas."

weakest = min(components, key=components.get)
```

3.4 Budget alerts (`services/budget_service.py`)

After a budget or an expense changes, the function compares each category's spend with its budget and writes one notification per threshold crossing. It skips a message that already exists, which prevents duplicates but also suppresses the same alert in a later month.

```
def check_and_raise_alerts(user_id: int, month: int, year: int) -> list[str]:
    created = []
    statuses = get_budget_status(user_id, month, year)
    with get_session() as session:
        for s in statuses:
            pct_used = s["percent_used"]
            if pct_used >= 100:
                msg = f"You have exceeded your {s['category']} budget."
                ntype = "danger"
            elif pct_used >= 90:
                msg = f"Your {s['category']} budget is 90% used."
                ntype = "warning"
            elif pct_used >= 75:
                msg = f"Your {s['category']} budget is 75% used."
                ntype = "warning"
            else:
                continue


            exists = session.query(Notification).filter(
                and_(Notification.user_id == user_id, Notification.message == msg)
            ).first()
            if not exists:
                session.add(Notification(user_id=user_id, message=msg, notification_type=ntype))
                created.append(msg)
    return created
```

3.5 Signed tokens (`backend/main.py`)

A token is a base64url JSON payload with the user id and an expiry, followed by an HMAC-SHA256 signature over that payload. Verification recomputes the signature, compares in constant time and checks the expiry. A tampered token was rejected in the smoke test.

```
def make_token(user_id: int) -> str:
    payload = _b64(json.dumps({"uid": user_id, "exp": int(time.time()) + TOKEN_TTL}).encode())
    sig = _b64(hmac.new(SECRET, payload.encode(), hashlib.sha256).digest())
    return f"{payload}.{sig}"

def read_token(token: str) -> int | None:
    try:
        payload, sig = token.split(".", 1)
        good = _b64(hmac.new(SECRET, payload.encode(), hashlib.sha256).digest())
        if not hmac.compare_digest(sig, good):
            return None
        data = json.loads(base64.urlsafe_b64decode(payload + "=" * (-len(payload) % 4)))
        return int(data["uid"]) if data["exp"] > time.time() else None
    except Exception:
        return None
```

3.6 Assistant routing (`services/ai_service.py`)

The question passes through five deterministic handlers in a fixed order. The first that matches answers from the user's data and the answer is tagged analytics; otherwise the optional language-model layer is tried, and finally a fixed help text is returned.

```
for handler in (
    lambda: _answer_can_i_save(question, user_id, monthly_income),
    lambda: _answer_reduce_expenses(question, user_id, monthly_income),
    lambda: _answer_financial_health(question, user_id, monthly_income),
    lambda: _answer_where_overspending(question, user_id),
    lambda: _answer_food_or_category_spend(question, user_id),
):
    result = handler()
    if result:
        return {"answer": result, "disclaimer": DISCLAIMER, "source": "analytics"}
```

4. IMPLEMENTATION DECISIONS AND THEIR REASONS

The main implementation decisions, the reason for each and what the measurements later showed are listed in the table below.

Table 8: Implementation decisions and their reasons

Decision	Reason	What the measurements showed
SQLite file with SQLAlchemy	No server to install; the same models run the Streamlit and web front ends	Dashboard 67 ms and transactions list 13 ms median; no concurrency test, no migrations
FastAPI layer over unchanged services	The business logic stays in one place and both front ends call it	33 routes with no logic of their own; 49 of 49 smoke checks passed
TF-IDF + logistic regression, not BERT	Runs on a CPU in milliseconds with no download; every weight is inspectable	71.7 percent on the probe set, 18.2 percent on unseen merchants; 0.84 ms per prediction
Keyword table as fallback	The app keeps working if the pickle cannot be loaded	Alone it scores 51.5 percent, so it is a safety net, not a second model
Retrain on every correction	Cheap enough to do inline and gives visible adaptation	Retrain 5.2 ms on the seed; repeat merchants 100 percent, new merchants barely move
Least-squares line for the forecast	Needs only three months and is easy to explain	MAPE 28.4 percent on the backtest against 21.4 percent for the mean; better only after about seven months in simulation
Weighted health score with fixed weights	Transparent and explainable with a named weakest area	Mean 60.3 over 5,000 random profiles; no fitted validation
Rule-routed assistant before any LLM	Answers are computed from the user's data and cannot invent figures	14 of 20 answerable questions handled; the LLM layer was not tested
bcrypt through passlib, pinned to 4.0.1	Slow salted hashing; bcrypt 5 breaks passlib	Login costs 305 ms by design
Home-made HMAC token with seven-day expiry	No extra dependency; stateless	No refresh or revocation; default secret if the environment variable is unset
Import commits row by row	Simple code and per-row warnings	35.2 rows per second; preview of 100 rows 667 ms
Reports generated on request	No stored files	CSV 18 ms, PDF 136 ms, Excel 209 ms median

Three decisions deserve a longer note. The first is the choice of a small categoriser. A fine-tuned BERT model would almost certainly do better on unseen text, but it needs labelled data the team did not have, a download of several hundred megabytes and far more time per prediction. The measurement shows the price of the small model: it is accurate on wording it has seen and poor on wording it has not, and it fails silently. A confidence threshold would be the cheapest repair.

The second is to keep the service layer unchanged and wrap it in FastAPI. The original Streamlit application stays usable and every number on the web screens is computed by

the same function the Streamlit pages used. The cost is that the services open a database session per call, which is simple but is why import is slow.

The third is the assistant design. Answering from handlers that query the user's data means a stated amount always matches the database, which was checked for the Food total and the top category. The price is limited coverage, and the fallback text is the honest response to a question the router does not recognise.